

PROJECT 1: QUANTUM RANDOM NUMBER GENERATOR

Danielle Beck

Sofhia Chromiec

Zuzia Kuczynska

Professor: Rodrigo Pasti

Class: Quantum Computing

Location: Curitiba, Paraná, Brazil

Date: 2026

ABSTRACT

This academic project presents the design, implementation, and empirical analysis of a Quantum Random Number Generator (QRNG) utilizing quantum logic circuits. Unlike classical computational systems that rely on deterministic Pseudorandom Number Generators (PRNGs), quantum computing leverages inherent physical principles such as superposition and quantum state measurement to produce truly non-deterministic random numbers. In this report, two primary scenarios are modeled to generate integer values bounded within the range $[0, 1000]$; a uniform distribution using standard Hadamard gates, and a biased distribution skewed toward higher integer values via custom phase and rotation gates (S , T , or R_z). Both quantum circuits were executed across ideal local simulators and real hardware on the IBM Quantum platform. A comparative analysis highlights the operational performance, noise impact, and distributional characteristics between classical PRNGs, noiseless quantum simulations, and physical Quantum Processing Units (QPUs).

Keywords: Quantum Computing, True Random Number Generation (TRNG), Qiskit, IBM Quantum, Quantum Circuits.

1. INTRODUCTION

Random number generation plays a fundamental role in cryptography, scientific simulations, financial modeling, and security protocol design. Classical computers generate pseudorandom numbers using mathematical algorithms (PRNGs) that, while statistically robust, are inherently deterministic if the initial state or seed value is known.

In contrast, quantum computing exploits the non-deterministic nature of quantum mechanics. When a qubit in a superposition state is measured, its state collapses probabilistically to either $|0\rangle$ or $|1\rangle$, producing genuine randomness

The objective of this project is to develop and evaluate two distinct Quantum Random Number Generators (QRNGs) capable of generating integer values in the range $[0, 1000]$:

1. **Uniform Distribution:** Every integer within the targeted interval has an equal probability of occurrence.
 2. **Biased Distribution:** The system is engineered using rotation or phase gates (S , T , or R_z) to introduce a controlled bias favoring higher numerical values.
-

2. UNIFORM QUANTUM RANDOM NUMBER GENERATOR

To map quantum bitstrings to the target integer range $[0, 1000]$, a minimum of 10 qubits ($2^{10} = 1024$) is required. Bitstrings representing integers greater than 1000 are discarded via rejection sampling to maintain a strict boundary condition.

2.1 Circuit Architecture

The uniform QRNG employs a Hadamard gate (H) across each active qubit to place the register into an equal superposition state:

$$|\psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$$

2.2 Conversion Methodology

1. **Register Preparation:** Initialize a 10-qubit quantum register and a corresponding 10-bit classical register.
2. **Superposition:** Apply H gates across all 10 qubits.
3. **Measurement:** Measure qubits in the computational basis to yield a binary string $B = b_9b_8\dots b_0$.
4. **Integer Mapping:** Convert binary B to decimal integer I . If $I > 1000$, re-sample.

Python

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import Aer
qc = QuantumCircuit(10, 10)
for q in range(10): qc.h(q)
qc.measure(range(10), range(10))
shots = 1000
backend = Aer.get_backend("qasm_simulator")
compiled_circuit = transpile(qc, backend)
run_out = backend.run(compiled_circuit, shots=shots).result()
out_counts = run_out.get_counts()
print("Raw quantum results:")
print(out_counts)
valid_counts = {}
```

```

for bitstring, count in out_counts.items():
    number = int(bitstring, 2)
    if number <= 1000:
        valid_counts[number] = count
print()
print("=====")
print(" UNIFORM RNG")
print("=====")
print(f"Number of shots: {shots}")
valid_shots = sum(valid_counts.values())
invalid_shots = shots - valid_shots
unique_values = len(valid_counts)
acceptance_rate = (valid_shots / shots) * 100
print(f"Valid shots: {valid_shots}")
print(f"Invalid shots: {invalid_shots}")
print(f"Acceptance rate: {acceptance_rate:.2f}%")
print(f"Unique values: {unique_values}")
if valid_counts:
    total = sum(number * count for number, count in
valid_counts.items())
    average = total / valid_shots
    print(f"Average: {average:.2f}")
    print(f"Minimum: {min(valid_counts)}")
    print(f"Maximum: {max(valid_counts)}")
else:
    print("No valid results were obtained.")

```

3. BIASED QUANTUM RANDOM NUMBER GENERATOR

To increase the probability of generating larger integers, single-qubit rotation and phase gates are applied:

$$R_y(\theta)R_y(\theta)=R_y(2\theta), R_z(\theta)R_z(\theta)=R_z(2\theta)$$

along with the phase gates:

$$S=[100i], T=[100e^{i\pi/4}].$$

These operations modify the measurement probabilities of the qubits. In the implemented circuit, the same transformation is applied independently to all ten qubits, increasing the probability of measuring the $|1\rangle$ state to approximately:

$$P(|1\rangle)\approx 0.80$$

Because the transformation is applied across the entire ten-qubit register, the resulting bitstrings contain more **1s** on average, causing the generated binary values to concentrate toward higher numerical ranges.

3.2 Circuit Design and Implementation

Python

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import Aer
import math
BIAS = 0.80
theta = 2 * math.asin(math.sqrt(BIAS))
qc = QuantumCircuit(10, 10)
```

```

for q in range(10):
    qc.h(q)
    qc.s(q)
    qc.t(q)
    qc.rz(theta - 3 * math.pi / 4, q)
    qc.h(q)
qc.measure(range(10), range(10))
shots = 1000
backend = Aer.get_backend("qasm_simulator")
compiled_circuit = transpile(qc, backend)
run_out = backend.run(compiled_circuit, shots=shots).result()
out_counts = run_out.get_counts()
print("Raw quantum results:")
print(out_counts)
valid_counts = {}
for bitstring, count in out_counts.items():
    number = int(bitstring, 2)
    if number <= 1000:
        valid_counts[number] = count
print()
print("=====")
print(" BIASED RNG")
print("=====")
print(f"Number of shots: {shots}")
valid_shots = sum(valid_counts.values())
unique_values = len(valid_counts)

```

```

invalid_shots = shots - valid_shots
acceptance_rate = (valid_shots / shots) * 100
print(f"Valid shots: {valid_shots}")
print(f"Invalid shots: {invalid_shots}")
print(f"Acceptance rate: {acceptance_rate:.2f}%")
print(f"Unique values: {unique_values}")
if valid_counts:
    total = sum(number * count for number, count in
valid_counts.items())
    average = total / valid_shots
    print(f"Average: {average:.2f}")
    print(f"Minimum: {min(valid_counts)}")
    print(f"Maximum: {max(valid_counts)}")
else:
    print("No valid results were obtained.")

```

4. QUANTUM SIMULATION AND IBM HARDWARE EXECUTION

Both quantum circuits were subjected to rigorous testing across two execution environments:

1. **Local Quantum Simulator:** High-performance, noiseless execution using Qiskit Aer (`qasm_simulator`).
2. **Physical IBM Quantum QPU:** Execution on a physical superconducting quantum computer via the IBM Quantum Platform.

4.1 Execution Setup and Parameters

- **shots per Execution:** 1,000 shots
- **Local Simulator:** Qiskit Aer (`qasm_simulator`)
- **IBM Backend:** `ibm_marrakesh`
- **IBM Execution:** Physical IBM Quantum QPU
- **Optimization Level:** Level 1 transpilation

4.2 Circuit Design and Implementation

Python

```
from qiskit import QuantumCircuit
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2
from qiskit.transpiler import generate_preset_pass_manager
import math

# =====
# IBM QUANTUM CONNECTION
# =====

service = QiskitRuntimeService()
backend = service.least_busy(operational=True,
                             simulator=False, min_num_qubits=10)
print("IBM backend:", backend.name)
print("Number of qubits:", backend.num_qubits)

# =====
# UNIFORM RNG CIRCUIT
# =====

qc = QuantumCircuit(10)
```



```

for q in range(10): qc.h(q)
qc.measure_all()

# =====
# BIASED RNG CIRCUIT
# =====

BIAS = 0.80
theta = 2 * math.asin(math.sqrt(BIAS))
qc1 = QuantumCircuit(10)
for q1 in range(10):
    qc1.h(q1)
    qc1.s(q1)
    qc1.t(q1)
    qc1.rz(theta - 3 * math.pi / 4, q1)
    qc1.h(q1)
qc1.measure_all()

# =====
# TRANSPILATION
# =====

print()
print("Transpiling circuits for", backend.name, "...")
print()
pass_manager = generate_preset_pass_manager(backend=backend,
optimization_level=1)
qc_isa = pass_manager.run(qc)

```

```

qc1_isa = pass_manager.run(qc1)
print("Uniform circuit transpiled.")
print("Biased circuit transpiled.")

# =====
# RUN BOTH CIRCUITS ON IBM
# =====

shots = 1000
sampler = SamplerV2(mode=backend)
print()
print("Submitting circuits to IBM Quantum...")
print()
job = sampler.run([qc_isa, qc1_isa], shots=shots)
print("Job ID:")
print(job.job_id())
print()
print("Waiting for IBM Quantum results...")
print()
result = job.result()

# =====
# EXTRACT RESULTS
# =====

uniform_data = result[0].data.meas
biased_data = result[1].data.meas
uniform_counts = uniform_data.get_counts()

```

```

biased_counts = biased_data.get_counts()

# =====
# ANALYSIS FUNCTION
# =====

def process_results(counts, shots, title):
    valid_counts = {}
    for bitstring, count in counts.items():
        number = int(bitstring, 2)
        if number <= 1000:
            valid_counts[number] = count

    print()
    print("=====")
    print(f" {title}")
    print("=====")
    print(f"Number of shots: {shots}")
    valid_shots = sum(valid_counts.values())
    unique_values = len(valid_counts)
    acceptance_rate = (valid_shots / shots) * 100
    print(f"Valid shots: {valid_shots}")
    print(f"Unique values: {unique_values}")
    print(f"Acceptance rate: {acceptance_rate:.2f}%")
    if valid_counts:
        total = sum(number * count for number, count in
valid_counts.items())
        average = total / valid_shots

```

```

        variance = sum(count * (number - average) ** 2 for
number, count in valid_counts.items()) / valid_shots
        standard_deviation = math.sqrt(variance)
        print(f"Average: {average:.2f}")
        print(f"Standard deviation: {standard_deviation:.2f}")
        print(f"Minimum: {min(valid_counts)}")
        print(f"Maximum: {max(valid_counts)}")
    return valid_counts

# =====
# DISPLAY RAW RESULTS
# =====

print()
print("RAW UNIFORM RESULTS:")
print(uniform_counts)
print()
print("RAW BIASED RESULTS:")
print(biased_counts)

# =====
# ANALYZE RESULTS
# =====

process_results(uniform_counts, shots, "UNIFORM RNG")
process_results(biased_counts, shots, "BIASED RNG")

```

5. RESULTS AND COMPARATIVE ANALYSIS

This section compares the statistical distributions obtained from the classical pseudorandom number generator, the local noiseless quantum simulator, and the physical IBM Quantum hardware. Each execution consisted of 1,000 shots, with results outside the interval [0,1000] excluded from the valid sample.

5.1 Distribution Comparison Table

Environment	Scenario	Mean	Std. Deviation	Min–Max	Acceptance
Classical PRNG	Uniform	499.73	282.06	0–1000	97.63%
Local Quantum Simulator	Uniform	495.49	290.37	0–1000	97.77%
IBM Quantum Hardware	Uniform	500.69	287.09	0–1000	97.50%
Classical PRNG	Biased	728.96	237.89	31–1000	69.00%
Local Quantum Simulator	Biased	726.99	233.11	39–999	69.70%
IBM Quantum Hardware	Biased	711.08	242.70	38–1000	71.80%

Table note: *Values represent the arithmetic mean of the repeated executions for each environment (each environment was run at least 3 times). Minimum and maximum values show the overall observed bounds across the corresponding runs.*

The source results show three classical and three local quantum executions, while the IBM dataset contains two executions. The individual runs consistently show the same overall pattern between environments.

5.2 Comparative Analysis

Uniform Scenario Analysis:

The uniform scenario produced results close to the expected midpoint of the target interval, 500500, across all three execution environments. The classical PRNG obtained an average of **499.73**, the local quantum simulator obtained **495.49**, and the IBM hardware obtained **500.69**.

The differences from the theoretical midpoint were therefore small: approximately **0.27**, **4.51**, and **0.69**, respectively. This indicates that all three approaches reproduced the expected central tendency of a uniform distribution reasonably well.

The acceptance rates were also highly consistent, ranging from **97.50% to 97.77%**. This is close to the theoretical acceptance probability of:

$$\frac{1001}{1024} \times 100 \approx 97.75$$

The similarity between the classical and quantum results demonstrates that, at the statistical level measured in this experiment, all three systems can generate samples with characteristics consistent with the intended uniform distribution.

Biased Scenario Analysis:

The biased scenario was designed with a probability of approximately:

$$P(|1\rangle) = 0.80$$

for each qubit, causing the resulting bitstrings to contain more 1s and therefore favor larger integer values.

The results clearly show this shift. The classical PRNG produced an average value of 728.96, while the local quantum simulator produced 726.99. The IBM hardware produced a lower average of 711.08.

The local simulator therefore closely reproduced the behavior of the classical implementation, while the IBM hardware showed a weaker effective bias toward higher values. This difference is particularly relevant because the local simulator represents the intended circuit behavior without physical hardware imperfections.

The acceptance rates also reflect the effect of the bias. They were approximately 69.00% for the classical implementation, 69.70% for the local simulator, and 71.80% for IBM hardware. These values are substantially lower than the uniform case because the biased circuit produces more bitstrings close to the upper end of the 10-bit range, increasing the likelihood of values above 1000 being discarded.

Hardware Noise & Decoherence Effects:

The comparison between the local noiseless simulator and the physical IBM Quantum processor demonstrates measurable differences in the observed distributions.

For the uniform scenario, the local simulator produced a mean of **495.49**, compared with **500.69** on IBM hardware. For the biased scenario, the difference was more pronounced: **726.99** on the simulator versus **711.08** on IBM hardware.

These deviations are consistent with the effects of **physical noise and other imperfections in current quantum hardware**, including gate errors and imperfect qubit operations. The results therefore demonstrate that a physical QPU does not necessarily reproduce the ideal simulated distribution exactly.

However, the experiment does **not directly measure decoherence itself**. Therefore, the results should be interpreted as showing differences consistent with hardware noise and physical imperfections rather than as a direct measurement of decoherence.

6. CONCLUSION

This project implemented and evaluated two Quantum Random Number Generators for producing integers in the range [0,1000]: a uniform generator based on Hadamard gates and a biased generator designed to favor higher values using S, T, and Rz gates. Both circuits generated 10-bit quantum measurement outcomes, with values above 1000 excluded from the valid samples.

Across the experiments, the local quantum simulator produced distributions consistent with the intended circuit designs. The uniform generator achieved an average of **495.49**, close to the theoretical midpoint of 500, while the biased generator achieved an average of **726.99**, demonstrating the intended shift toward higher values. The classical PRNG produced similar statistical results, with means of **499.73** and **728.96**, respectively.

Execution on the physical IBM Quantum hardware also reproduced the general behavior of both

circuits, but with measurable differences from the noiseless simulator. In particular, the biased IBM results showed a lower average value than the simulator, indicating the influence of physical hardware imperfections.

Overall, the experiment demonstrates that quantum measurement can be used as a physical source of randomness and that the implemented circuits successfully reproduced both the uniform and intentionally biased distributions on a simulator and a real quantum processor

REFERENCES

1. NIELSEN, Michael A.; CHUANG, Isaac L. **Quantum Computation and Quantum Information**. 10th Anniversary ed. Cambridge: Cambridge University Press, 2010.
2. IBM QUANTUM. **Qiskit Documentation and Developer Guide**. Available at: <https://docs.quantum.ibm.com/>. Accessed: 2026.
3. HERRERO-COLLANTES, Miguel; GARCIA-ESCARTIN, Juan Carlos. Quantum random number generators. **Reviews of Modern Physics**, v. 89, n. 1, p. 015004, 2017.
4. Project Repository: https://github.com/DaniaB-sec/Qistik_Simulations.git